

Gestione degli errori

Errori comuni

Come riconoscere gli errori PHP più frequenti e leggerne i messaggi.

Leggere un errore

Un messaggio di errore di solito dice tre cose:

1. che tipo di errore è
2. in quale file è successo
3. a quale riga guardare

La riga indicata non è sempre la causa esatta, ma è un ottimo punto da cui partire.

Parse error

Un parse error significa che PHP non riesce a capire la sintassi.

```
<?php  
echo "Ciao"
```

Qui manca il punto e virgola. PHP può mostrare un errore vicino alla riga successiva, perché si accorge tardi del problema.

Warning

Un warning segnala un problema, ma spesso il programma continua.

```
<?php  
include "file-che-non-esiste.php";
```

PHP avvisa che non trova il file.

Notice o warning su variabili mancanti

```
<?php  
echo $nome;
```

Se `$nome` non è stata definita, PHP segnala il problema. Prima di usare una variabile, assicurati che abbia un valore.

```
<?php  
$nome = $_GET["nome"] ?? "ospite";
```

TypeError

Un `TypeError` può comparire quando passi un tipo di dato sbagliato.

```
<?php  
function somma(int $a, int $b): int {  
    return $a + $b;  
}  
  
somma("ciao", 3);
```

La funzione si aspettava numeri interi, ma riceve una stringa.

Metodo pratico

Quando vedi un errore:

1. leggi il tipo di errore
2. apri il file indicato
3. guarda la riga indicata e quella prima
4. controlla parentesi, virgolette e punti e virgola

5. riduci il codice finche il problema diventa visibile

Esempio di lettura del messaggio

Un messaggio puo assomigliare a questo:

```
Parse error: syntax error, unexpected token "echo" in index.php on line 5
```

Leggilo pezzo per pezzo:

- `Parse error` indica un problema di sintassi
- `unexpected token "echo"` dice che PHP ha trovato `echo` dove non se lo aspettava
- `index.php` e il file
- `line 5` e la riga da controllare

Spesso l'errore vero e nella riga precedente: un punto e virgola mancante, una parentesi non chiusa o una virgoletta dimenticata.

Warning non significa "ignorami"

Un warning puo lasciare continuare il programma, ma va comunque capito. Se PHP avvisa che un file manca o che una variabile non esiste, il risultato della pagina potrebbe essere sbagliato.

Prova tu

Crea apposta un piccolo errore, per esempio toglì un punto e virgola. Leggi il messaggio e prova a trovare file e riga. Poi correggi e riesegui il file.

Debugging

Come osservare cosa fa il codice PHP e trovare problemi passo dopo passo.

Cosa significa debugging

Fare debugging significa cercare e correggere un problema nel codice. Non e indovinare: e osservare con ordine.

var_dump

`var_dump` mostra valore e tipo.

```
<?php
$eta = "25";
var_dump($eta);
```

Output possibile:

```
string(2) "25"
```

Questo ti dice che `$eta` sembra un numero, ma in realta e una stringa.

print_r

`print_r` e comodo per array.

```
<?php
$utente = ["nome" => "Luca", "eta" => 28];
print_r($utente);
```

Output:

```
Array
(
    [nome] => Luca
    [eta] => 28
)
```

Mostrare gli errori durante lo studio

In ambiente di sviluppo puoi attivare la visualizzazione degli errori.

```
<?php
error_reporting(E_ALL);
ini_set('display_errors', '1');
```

Attenzione: su un sito pubblico non mostrare errori tecnici agli utenti. Possono rivelare dettagli interni del progetto.

Log degli errori

PHP può scrivere errori nei log del server. I log sono file dove vengono registrati problemi e informazioni utili.

Quando lavori su un hosting o un server, cerca la sezione dedicata agli error log.

Un metodo ordinato

Quando qualcosa non funziona:

1. riproduci il problema
2. controlla il messaggio di errore
3. stampa le variabili importanti con `var_dump`
4. verifica una riga alla volta cosa succede
5. toglì le stampe di debug quando hai finito

Il debugging migliora con la pratica. L'importante è fare un passo alla volta.

Restringere il punto del problema

Quando una pagina è lunga, non cercare tutto insieme. Metti piccole stampe in punti strategici.

```
<?php
echo "Prima del calcolo\n";

$totale = $prezzo * $quantita;

echo "Dopo il calcolo\n";
var_dump($totale);
```

Se vedi il primo messaggio ma non il secondo, il problema è tra i due punti.

Controllare i dati ricevuti

Molti problemi nascono da dati diversi da quelli che immagini.

```
<?php
var_dump($_POST);
```

Questa stampa ti mostra tutti i campi arrivati dal form. Usala solo durante lo sviluppo e rimuovila quando hai finito.

Diario del bug

Per problemi difficili, scrivi tre righe:

1. cosa mi aspettavo
2. cosa succede davvero
3. quale valore controllo adesso

Questo ti impedisce di cambiare codice a caso. Il debugging diventa una piccola indagine ordinata.

Eccezioni

Come gestire situazioni impreviste con try, catch e throw.

Cosa sono le eccezioni

Un'eccezione segnala che qualcosa è andato storto e il flusso normale non può continuare.

Non tutti gli errori sono uguali. Un file mancante può essere una situazione prevista da gestire. Una variabile scritta male è invece un bug da correggere.

try e catch

Metti nel blocco `try` il codice che potrebbe fallire. Nel blocco `catch` gestisci il problema.

```
<?php
try {
    $pdo = new PDO($dsn, $utente, $password);
    echo "Connessione riuscita";
} catch (PDOException $errore) {
    echo "Connessione non riuscita";
}
```

Se la connessione fallisce, PHP passa al `catch`.

throw

Puoi lanciare tu un'eccezione quando trovi una situazione non valida.

```
<?php
function dividi(float $a, float $b): float {
    if ($b === 0.0) {
        throw new InvalidArgumentException("Non puoi dividere per zero");
    }

    return $a / $b;
}
```

finally

`finally` viene eseguito comunque, sia se tutto va bene sia se c'è un'eccezione.

```
<?php
try {
    echo "Apro una risorsa";
} catch (Exception $errore) {
    echo "Errore";
} finally {
    echo "Pulizia finale";
}
```

Errore previsto o bug?

Usa le eccezioni per casi che il programma può incontrare e gestire: connessione fallita, file mancante, dato non valido.

Correggi invece i bug alla radice: nomi sbagliati, parentesi mancanti, logica errata. Un `catch` non deve nascondere codice rotto.

Recuperare il messaggio dell'eccezione

L'oggetto ricevuto nel `catch` contiene informazioni sull'errore.

```
<?php
try {
    throw new RuntimeException("File non trovato");
} catch (RuntimeException $errore) {
    echo $errore->getMessage();
}
```

Output:

```
File non trovato
```

Durante lo studio questo è utile. In produzione, invece, evita di mostrare dettagli tecnici agli utenti.

Eccezioni personalizzate

Per programmi piccoli puoi usare le eccezioni già pronte, come `RuntimeException` o `InvalidArgumentException`. Nei progetti più grandi puoi creare eccezioni con nomi specifici.

```
<?php
class ProdottoNonDisponibile extends RuntimeException {
}
```

Il nome rende più chiaro che tipo di problema stai gestendo.

Errore comune: catturare tutto e continuare

Un `catch` non deve far finta che vada tutto bene. Se non sai gestire davvero l'errore, registra il problema o mostra un messaggio chiaro. Nascondere l'errore rende il programma più difficile da correggere.